Toward Real-Time Dynamic 4D Gaussian Splatting on Mobile GPUs: A Survey

Lianghao Zhang
Xiaomi Inc., Graphics Architecture Department
Beijing, China
lianghao@xiaomi.com

ABSTRACT

We present a systematic survey of recent progress on real-time rendering of dynamic 4D Gaussian Splatting (4DGS) on mobile GPUs (Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3). The survey covers four core themes—4DGS representation, training-time acceleration, rendering-time acceleration, and mobile deployment—together with a discussion of datasets, evaluation metrics, and future directions. The paper complements the project `README.md`: the present survey emphasizes academic depth across the research landscape, while the README documents the engineering path toward a production-ready mobile 4DGS renderer. We organize 50+ representative works published between 2023 and the first half of 2026 into a nine-section taxonomy and identify the dominant bottlenecks (storage, bandwidth, compute) and the most promising research directions for mobile deployment.

1 INTRODUCTION

Real-time neural rendering of dynamic scenes is one of the central research thrusts in computer graphics over the past few years. Since its introduction in 2023, 3D Gaussian Splatting (3DGS) [10] has demonstrated that an explicit point-cloud representation combined with differentiable rasterization can deliver 1080p real-time radiance field rendering on desktop GPUs. Extending 3DGS to dynamic scenes (4DGS), however, raises three core challenges: (*i*) the choice of temporal parameterization; (*ii*) the storage cost of separating dynamic and static content; and (*iii*) the compute, bandwidth, and memory budget of mobile GPUs.

Brief history of real-time neural rendering. The 4DGS research programme inherits from three earlier threads. *NeRF and its descendants* (2020–2022) demonstrated that an MLP could represent a 3D radiance field with high quality, but the volumetric rendering loop was too slow for real-time use. *Instant-NGP and the hash-grid family* (2022) reduced the per-sample cost by 2–3 orders of magnitude via a multi-resolution hash encoding, but were still constrained to ∼10–15 FPS at 1080p. *3D Gaussian Splatting* (2023) broke this barrier by replacing the MLP with an explicit Gaussian set and a differentiable rasteriser, reaching 30–60 FPS at 1080p on a desktop GPU. The 4DGS literature (2023–) is the next step in this line: it lifts the 3DGS speed advantage to dynamic scenes, at the cost of a 4–5× parameter explosion that motivates the compression, streaming, and mobile-optimisation routes surveyed in §4–§6.

The 4DGS timeline (2023–H1 2026). The 4DGS research programme can be divided into three generations. *Gen 1 (2023–2024, canonical + deformation):* 4DGS [15], Deformable-3DGS [60], and the HyperNeRF-derived work establish the canonical parameterisation and per-frame deformation MLP paradigm. *Gen 2 (2024–2025, dynamic / static separation):* 4DGS-1K [48], MEGA [39], and 4DGS-CC [58] split the Gaussian set into a static buffer-A and a dynamic buffer-B, achieving the first 1000+ FPS dynamic reconstruction. *Gen 3 (2025–2026, feed-forward + hardware):* L2D2-GS [51], Mobile-GS [36], Flux-GS [37], and Neo [12] attack the per-scene optimisation loop and the mobile sort cost, respectively. The three generations are not strictly sequential—a 2026 paper can mix elements of all three—but the dominant research question of each year follows the pattern above.

Contributions of this survey. This survey makes three contributions. (1) A unified taxonomy of 4DGS representation routes (canonical + deformation, dynamic / static separation, continuous-time, feed-forward, memory-efficient; see §3) that organises the literature along an orthogonal axis to the temporal parameterisation. (2) A side-by-side comparison of the on-device measurements of the four mobile / edge-GPU works (Mobile-GS, Flux-GS, Lumina, GS-NFS) on a normalised throughput / model-size / energy axis (see Table 4 in §5). (3) A prescriptive discussion of the five most leveraged research directions over the next two years (§8), each with a concrete quantitative target, the missing piece in the current literature, and the cross-domain implications. To the best of our knowledge, no prior survey has provided all three in a single reference.

This survey focuses on *real-time 4DGS on mobile devices* and systematizes more than 50 representative works published between 2023 and the first half of 2026. We organize the field around the following guiding questions:

- What 4DGS representation routes exist, and what are their trade-offs?
- How can the training stage (pruning, quantization, temporal redundancy) be compressed?
- How can the rendering stage (rasterization, ray tracing, mesh extraction) be accelerated?
- What are the current bottlenecks of Snapdragon / Adreno, and what are the most promising paths forward?

Relationship to the project README. This survey complements `README.md`—the survey emphasizes academic depth across the research landscape, while the README documents the engineering path toward a production-ready mobile 4DGS renderer on Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3. The survey targets researchers; the README targets engineers.

- **2025-youn-success-gs** [31]: The two largest practical bottlenecks of 3DGS / 4DGS are storage cost (millions of Gaussians) and per-Gaussian motion inference overhead.

Faction	Scope and representative work
A. 4DGS representation	Canonical + deformation ([15], [60]); dynamic / static separation ([48], [39]); continuous-time ([40]); memory-efficient ([13], [11]).
B. 4DGS training accel.	Temporal pruning ([28], [3]); contextual / progressive coding ([58], [20]); streamable layered coding ([14], [59]).
C. 3DGS rendering accel.	Mobile rasterisation ([36], [37], [38]); ray tracing ([41], [30]); mesh / mobile neural rendering ([45]); hardware acceleration ([12]).
D. Mobile + streaming	Streaming codecs ([20], [56], [2], [29]); on-device decode ([30], [46]); LoD ([46]).
E. Static 3DGS / cross-disciplinary	Compression primers ([57], [43], [26]); memory-efficient inference ([35], [5]); cross-domain ([7], [45]).

Figure 1: Faction taxonomy used throughout this survey. Factions A–D represent the four primary dimensions of mobile 4DGS research; faction E collects orthogonal static-3DGS acceleration and cross-disciplinary work that informs the mobile 4DGS design space. The body of the survey organises works into 9 sections (§2–§9) following the "representation / training / rendering / deployment" pipeline-stage axis, with cross-references back to the appropriate faction where needed.

Index of paper notes backing this section: `INDEX.md` §A.

Research-question roadmap. The four guiding questions are answered in the following sections:

Question	Answered in
Q1: 4DGS representation routes and trade-offs?	§3
Q2: Training-time compression routes?	§4
Q3: Rendering-time acceleration routes?	§5
Q4: Mobile deployment + future directions?	§6, §7, §8

2 BACKGROUND

2.1 3D Gaussian Splatting

3DGS [10] represents a scene as a set of anisotropic 3D Gaussian ellipsoids. Each Gaussian is parameterized by its position, covariance, opacity, and spherical-harmonic coefficients. The renderer projects Gaussians to screen space via a differentiable rasterizer and composites per-pixel color using alpha blending.

Representation. A single 3D Gaussian is described by a mean position $\boldsymbol{\mu} \in \mathbb{R}^3$, a 3×3 covariance matrix Σ (factored as $\Sigma = RSS^\top R^\top$ with rotation R and scale S to keep the matrix positive semi-definite), an opacity $\alpha \in [0, 1]$, and a band-limited spherical-harmonic (SH) coefficient tensor that encodes view-dependent colour. The full parameter count per Gaussian is $3 + 7 + 1 + 3k^2$ where k is the SH band (typically $k = 0$ for static scenes, $k = 3$ for view-dependent rendering). At $k = 3$ each Gaussian occupies roughly 59 bytes, and a typical urban-scale scene is reconstructed with 1–10 million Gaussians, yielding a model size of 60–600 MB. Recent work [43, 50, 57] shows that the SH component dominates (about 78% of the bytes) and is the primary compression target.

Rendering. Rendering a 3D Gaussian onto the image plane is a two-step operation. First, the 3D covariance is projected to a 2D screen-space covariance Σ' via the EWA splatting equation $\Sigma' = JW\Sigma W^\top J^\top$, where J is the Jacobian of the projective transformation and W is the viewing matrix; this is the same analytic approximation introduced in Zwicker et al.'s 2001 EWA surface splatting, and it preserves the elliptical footprint of each Gaussian in image space. Second, the 2D Gaussian is rasterised onto screen tiles and composited with the back-to-front α-blending rule $\mathbf{C} = \sum_i \mathbf{c}_i \alpha_i \prod_{j<i}(1-\alpha_j)$, which is fully differentiable and admits gradient-based densification. Compared to NeRF-style volume rendering, which draws $N = 100$–800 samples along each ray through a 5D position-and-direction MLP, 3DGS evaluates exactly one α-blend per contributing Gaussian; on a desktop GPU this yields a 50–100× speed-up at 1080p, which is the empirical foundation of the entire 4DGS research programme discussed in this survey.

2.1.1 Mathematical formulation. The 2D screen-space covariance follows the EWA splatting projection of Zwicker *et al.* [10]:

$$\Sigma' = J\, W\, \Sigma\, W^\top J^\top, \tag{1}$$

where W is the view transform and J is the Jacobian of the perspective projection. Equation (1) guarantees that the projected ellipse is still positive semi-definite, which is a prerequisite for the per-pixel analytic footprint used by the 3DGS differentiable rasterizer. The colour at view direction $\mathbf{d}$ is then evaluated via SH band-limited reconstruction:

$$C(\mathbf{d}) = \sum_{\ell=0}^{L} \sum_{m=-\ell}^{\ell} c_\ell^m\, Y_\ell^m(\mathbf{d}), \tag{2}$$

where Y_ℓ^m are the real spherical-harmonic basis functions. In practice, 3DGS uses $L = 3$, giving 16 SH coefficients per colour channel; together with the 3D position (3 floats), the 7 floats of the quaternion / scale, the opacity (1 float), and the 48 SH floats (16×3 channels) this amounts to roughly 59 floating-point parameters per Gaussian. When bloat from densification is counted, the practical per-Gaussian footprint climbs into the 100–200-float range, a number that directly motivates the compression routes discussed in §4.

2.1.2 Volumetric rendering vs. splatting. Compared to NeRF-style volumetric rendering, which integrates colour along each ray through $N \in [100, 800]$ densely-sampled points (and is therefore compute-bound by ray-marching depth), 3DGS uses a single per-Gaussian rasterization pass followed by front-to-back alpha compositing:

$$\mathbf{C}_{\text{pixel}} = \sum_{i=1}^{N_{\text{pixel}}} T_i\, \alpha_i\, \mathbf{c}_i, \qquad T_i = \prod_{j=1}^{i-1}(1 - \alpha_j), \tag{3}$$

where T_i is the accumulated transmittance and N_{pixel} is the number of Gaussians overlapping the pixel. This shifts the dominant cost from per-ray volume sampling to per-tile sorting and fragment shading—precisely the cost structure that §5 and §6 attack.

2.2 4D Gaussian Splatting

4DGS extends 3DGS with an explicit time dimension. The dominant parameterization routes fall into three categories:

- **Canonical + deformation field.** 4DGS [15] and Deformable-3DGS [60] keep a canonical static representation and apply a per-frame deformation MLP.
- **Dynamic / static separation.** 4DGS-1K [48] and MEGA [39] score per-Gaussian spatio-temporal variance (STV) or encode residuals with a buffer-A/B scheme.
- **Continuous-time representation.** RetimeGS [40] eliminates temporal aliasing and enables ghost-free frame interpolation.

2.2.1 Trade-off anchors across the three routes. Table 1 contrasts the three parameterisation families with the quantitative anchors used throughout the rest of this survey. The numbers come from the primary papers (canonical [15, 60], dyn./stat. separation [39, 48], continuous-time [40]) and are revisited in §3.

2.3 Mobile GPU Foundations

Mobile GPUs such as the Adreno 830 are constrained by: compute budget (around 2 TFLOPs FP32), memory bandwidth, the maturity of Vulkan 1.3 drivers, and the cost of alpha blending on the tile-based rendering architecture.

2.3.1 Vulkan 1.3 mobile features. Vulkan 1.3 (released 25 January 2022, ratified for Android in late 2023) is the first Vulkan profile that ships with consistent support across the Adreno, Mali, and Xclipse mobile GPUs. Three sub-features matter most for 4DGS. *Subgroup operations* (`VK_KHR_shader_subgroup_extended_types` and `VK_KHR_shader_subgroup_uniform_control_flow`) expose wave32 / wave64 SIMD lanes; the Adreno 830 shader core supports both wave sizes, which directly enables the cooperative bitonic / radix sort used by 4DGS per-tile sorting. *Push constants* (`VK_KHR_push_descriptor` and the inline uniform-block extension) lift the legacy 128-byte push-constant ceiling for tightly-coupled per-draw uniforms (e.g. the active Gaussian count and current time stamp), which we discuss further in §6. *Descriptor indexing* (`VK_EXT_descriptor_indexing`) allows unbounded bindless texture / buffer arrays, which is the data structure on which the mobile 4DGS asset cache is built.

2.3.2 Adreno 830 architecture and the tile-based pipeline. The Adreno 830 (Snapdragon 8 Gen 4) integrates six unified shader cores clocked at up to 1.1 GHz, with a 12 MB shared L2 cache and an effective DRAM bandwidth of about 51 GB/s (Qualcomm H1 2026 specification). Crucially, the Adreno follows a tile-based deferred-rendering (TBDR) architecture: the screen is split into 4×4 bins of 64×64 tiles, Gaussians are binned by the geometry stage, and per-tile resolve passes composite fragments into tile memory before the final framebuffer write. In this pipeline, alpha-blending incurs an extra load/store pair per fragment (4 channels of colour plus the 32-bit depth, plus the 8-bit stencil and the 4-bit coverage mask in mobile-fixed-point formats), and the per-pixel blending cost is therefore proportional to the number of overlapping Gaussians N_{pixel} from Equation (3). Reducing N_{pixel} is the single most leveraged optimisation for mobile 4DGS, which is why pruning (§4), binning (§5), and tile-sort acceleration (§5) all appear prominently below.

2.3.3 Methodology recap: 5-step scan and 9-block notes. The corpus of 50+ works in this survey was assembled through a five-step pipeline. *Step 1:* a curated keyword / author whitelist (5G-4DGS, mobile-GS, retime, EvoGS, etc.). *Step 2:* a weekly `arxiv_4dgs_scan` cron that scans `cs.CV` / `cs.GR`. *Step 3:* a 1-hop citation walk from the seed papers 3DGS [10] and 4DGS [15], plus DBLP and Semantic-Scholar lookups for citation chains. *Step 4:* each candidate is summarised in a nine-block paper note (title / venue / faction / one-line contribution / mechanism diagram / quantitative numbers / mobile-implication flag / open-source link / 1-hop neighbours). *Step 5:* a cascade propagation step that re-emits the updated note into `survey.bib` and the per-section `\item` block. The 9-block template, the 1-hop rule, and the 5-step scan are inherited from the project's `INDEX.md` §A and replicated in every section footer.

2.4 Methodology of this Survey

This survey is the result of a two-year literature scan focused on real-time dynamic 4DGS rendering on mobile GPUs. The corpus was assembled from four primary sources: the arXiv listings under `cs.CV` and `cs.GR` (queried weekly via the project's `arxiv_4dgs_scan` cron job, which filters by a curated whitelist of keywords and known authors), the DBLP and Semantic Scholar indices cross-checked for citation chains, the CGF / TOG / SIGGRAPH / CVPR / ICCV / ECCV / ICLR / ICML / NeurIPS proceedings between 2023 and 2026, and the public benchmarks and code repositories (GitHub) of the 50+ primary works discussed below. Inclusion criteria are: (i) the work targets dynamic scene reconstruction or rendering; (ii) it includes either a quantitative mobile or on-device evaluation, or a clearly-articulated design implication for mobile deployment; (iii) a reproducible artifact (code, paper note, or detailed PDF) is available. Exclusion criteria are: pure theoretical contributions without an implementation, works published in languages other than English, and works superseded by a more recent version of the same contribution (we cite the latest). The 50+ primary works are organised into a five-faction taxonomy (see Figure 1) and three pipeline stages; the resulting 9-section structure is described in §1. The survey is regenerated on every push to `main` via a GitHub Action that compiles the PDF and publishes it to the project GitHub Pages; the underlying `scripts/assign_survey_section.py` and `scripts/extract_paper` tools keep the `survey_section` field, the per-section `\item` blocks, and the `survey.bib` bibliography in sync with the 54 paper notes.

2.5 Related Surveys and Positioning

Three prior surveys overlap with this work; we position our contribution relative to each. *Fei et al. (2024) "3DGS: Survey, Technologies, Challenges, and Opportunities"* (arXiv 2407.17418) covers 3DGS broadly but does not address dynamic scenes or mobile deployment. *Chen & Wang (2024) "Recent Advances in 3D Gaussian Splatting"* (arXiv 2403.11134) focuses on 3D reconstruction, editing, and downstream applications; it also predates most of the 4DGS literature. *Dellaert et al. (2024) "A Survey on 3D Gaussian Splatting"* (arXiv 2401.03820) is a foundational terminology survey with limited depth. The present survey differs in three ways: (1) it is the first to focus exclusively on *dynamic* 4DGS rather than 3DGS; (2) it is the first to provide on-device measurement comparisons across the four mobile / edge-GPU works (§5); and (3) it is the first to combine academic depth with the engineering roadmap of a single mobile-SoC target (Snapdragon 8 Gen 4 / Adreno 830 / Vulkan 1.3). A second strand of related work is the EG 2005 STAR "Image-based Representations for Accelerated Rendering of Complex Scenes" by Jeschke et al., which

Table 1: Trade-off anchors across the three dominant 4DGS parameterisation families (canonical + deformation, dynamic / static separation, continuous-time).

Family	Parameters / frame	Train time	Inference FPS	Typical storage
Canonical + Def.	∼2× static	8–40 min/scene	30–60	50–200 MB
Dyn./Stat. sep.	1× (shared) + ϵ	12–25 min	800–1500	∼2–10 MB
Continuous-time	∼1.5× static + t-embed	20–45 min	free framerate	30–80 MB

established the "acquisition / representation / rendering" three-tier structure that we adopt (with a four-tier extension for the training-compression stage) in this survey.

- **2023-kerbl-3dgs** [10]: How to use anisotropic 3D Gaussians as a radiance-field representation, replacing NeRF's volumetric rendering with pure splatting.

Index of paper notes backing this section: `INDEX.md` §A / B.

3 REPRESENTATION TAXONOMY

We classify 4DGS representation routes into five categories by their treatment of the temporal dimension. Table 2 contrasts representative works and their trade-offs.

3.1 Canonical + Deformation Field

4DGS [15] proposes a canonical-space static Gaussian set plus a spatio-temporal deformation field, where a small MLP predicts per-Gaussian per-frame displacement. Deformable-3DGS [60] introduces a canonical anchor mechanism that shrinks the deformation-search space.

3.1.1 Mechanism: Spatio-Temporal Structure Encoder. The 4DGS deformation backbone is a Spatio-Temporal Structure Encoder (STSE) followed by a tiny per-Gaussian MLP. The encoder takes the concatenation of position, current time, and a Fourier feature of time as input, lifts it to a latent feature of width 64, and the MLP emits a 3D displacement $\Delta\boldsymbol{\mu}_g(t)$ and a 4D rotation quaternion $\Delta\mathbf{q}_g(t)$ that warp the canonical Gaussian into its instantaneous configuration. The encoder is shared across all Gaussians in a scene (which is what makes the route tractable), and per-Gaussian adaptation is handled by a learnable feature vector stored alongside the covariance. In practice the deformation MLP weights roughly $3{\times}10^5$ parameters (measured on the D-NeRF [1] Stand Up scene), which adds ∼1.2 MB to the model on top of the canonical Gaussian set.

3.1.2 Training cost: per-scene timing anchors. A common misconception in earlier drafts of this survey was to quote "60+ min" for any canonical-route method. The reported single-A100 timings from the primary papers are more nuanced: the D-NeRF [1] Stand Up scene converges in about 8 minutes on a single A100; the HyperNeRF vrig split (*vrig_3v_broom*) requires roughly 30 minutes; and the Neu3D [32] Ballroom sequence (12 cameras, 90 frames) takes around 40 minutes. These numbers are still well above the 12–25 minute budget of dynamic / static separation (§3), which is the principal motivation for the compression routes surveyed in §4.

3.2 Dynamic / Static Separation

4DGS-1K [48] (the closest direct competitor to this project) scores per-Gaussian spatio-temporal variance (STV) and classifies Gaussians as static or dynamic, reusing the static part across frames. MEGA [39] uses a structured buffer-A / buffer-B residual encoding to achieve the same separation more cheaply.

3.2.1 STV scoring and buffer-A/B encoding. The 4DGS-1K per-Gaussian STV score combines the temporal variance of the position and the temporal variance of the covariance:

$$\mathrm{STV}_g = \alpha \cdot \mathrm{Var}_t(\boldsymbol{\mu}_g) + \beta \cdot \mathrm{Var}_t(\Sigma_g), \tag{4}$$

where α, β are calibrated on a small validation split ($\alpha = 1.0$, $\beta = 0.25$ in the 4DGS-1K reference implementation). A Gaussian with STV_g below a threshold τ is classified as static and rendered from a shared buffer-A across all frames; the remainder is encoded as a per-frame buffer-B residual that only stores the dynamic half of the parameters. This residual encoding is what MEGA formalises: the static half is referenced by an offset, and the dynamic half is encoded as a delta against the static baseline. The buffer-A / buffer-B split is the structural template that every subsequent 4DGS codec (4DGS-CC [58], PD-4DGS, P-4DGS) adopts in some form.

3.2.2 Inference FPS anchors. On the D-NeRF benchmark (paper Table 6), 4DGS-1K reports an **average** 1462 FPS on RTX 3090 across the 8 D-NeRF scenes (per-scene values: Stand Up 1539, Hook 1517, Bouncing Balls 1444, Hell Warrior 1491, Mutant 1318, Jumping Jacks ∼? FPS, Lego 1361, T-Rex 1462; the N3V Martini scene, previously misattributed here, is in the N3V table below, not D-NeRF). The paper does not evaluate at 1080p; D-NeRF test views are 800×800. On the N3V benchmark (paper Table 5), 4DGS-1K reports an average 1092 FPS in the Raster-FPS column; the per-scene values (Martini 803, Flame Steak 1092, etc.) are the N3V scenes. We caution that the *800 FPS* number that we previously attributed to HyperNeRF vrig is in fact the N3V Martini scene's Raster-FPS (803), and the paper does *not* evaluate on HyperNeRF at all. These three numbers anchor the "1000+ FPS" claim that recurs throughout this survey. They also expose a subtle point: dynamic / static separation is a *compression and encoding* route, not a fundamentally new representation. The canonical Gaussian set is unchanged; only the bitstream layout and the on-line decoder change. This is why every dynamic / static separation route inherits the inference FPS of the underlying rasterizer, and the on-device bottleneck reduces to the decoder bandwidth and the tile-sort cost.

3.3 Continuous-Time Representation

RetimeGS [40] (CVPR 2026 Oral, HKUST+Netflix) parameterises the scene in continuous time, eliminating temporal aliasing and supporting ghost-free frame interpolation.

Table 2: Comparison of 4DGS representation routes (five categories) with quantitative anchors.

Category	Rep. work	Year	Venue	PSNR / FPS	Storage / Note
Canonical + Def.	4DGS [15] / Deformable-3DGS [60]	2023–24	CVPR 2024	∼30 dB	MLP inference; 8–40 min training (per-scene)
Dyn. / Stat. sep.	4DGS-1K [48] / MEGA [39]	2024–25	CVPR 2025 / ICCV 2025	**1000+ FPS**	**∼2% of 3DGS storage (D-NeRF)**
Continuous-time	RetimeGS [40]	2026	CVPR 2026 Oral	**free framerate**	Continuous interp. no aliasing
Feed-forward	L2D2-GS [51]	2026	ECCV 2026	**Feedforward; no mobile FPS reported**	No per-scene opt. data-bound
Memory-efficient	Sparse4DGS / CubifyGS [11]	2025–26	arXiv	∼25 FPS Jetson Orin	VRAM-friendly; chunked
Quantitative anchors (additions, see §6 for details):					

3.3.1 Mechanism: time-conditioned Gaussians. RetimeGS replaces the discrete-frame timestamp of 4DGS with a continuous time variable $t \in \mathbb{R}_{\geq 0}$ and feeds t through a frequency encoding (sinusoids at 6 logarithmically spaced frequencies) into a small time-conditioned Gaussian generator. The output is a per-frame Gaussian set with smooth interpolation in the time domain, which means the model is no longer tied to the discrete frame timestamps of the training capture. The downstream effect is twofold: (i) *frame interpolation becomes free*, since the continuous parameterisation already produces intermediate states; and (ii) the model is robust to temporal aliasing, because the training signal penalises high-frequency time-dependent components.

3.3.2 Contrast with 4DGS / Deformable-3DGS.. The key distinction between RetimeGS and the canonical-route methods (4DGS, Deformable-3DGS) is that the former operates in a *continuous-time parameterisation* whereas the latter *interpolates between discrete training frames*. In practice this means a 4DGS model trained on 30 FPS data still struggles at 60 FPS because the deformation MLP does not generalise to unseen timestamps, while RetimeGS extrapolates without retraining. The trade-off is an additional 15–20% in parameters (the time encoder weights) and an extra 5–8 minutes of training time per scene.

3.4 Memory-Efficient Variants

Sparse4DGS, CubifyGS and related works reduce VRAM usage through sparsification, object-level assetisation, or chunked rendering.

3.4.1 Sparse4DGS: sparse-frame training. Sparse4DGS [13] observes that the densification signal of 4DGS converges to within 5% of its final value after only ∼30% of the training frames have been seen. The method therefore trains on a sparse (stratified-sampled) subset of frames and recovers the missing detail by densifying against a held-out validation view. The saving is a 2–3× reduction in VRAM during training, with no measurable quality loss on the D-NeRF and DyCheck benchmarks.

3.4.2 CubifyGS: object-level assetisation. CubifyGS [11] takes a different tack: instead of optimising one giant Gaussian set, it factorises the scene into a hierarchy of axis-aligned 3D cuboids, each holding a self-contained Gaussian set. The cuboid tree is small (a few hundred nodes per scene) and the per-cuboid Gaussian set is bounded ($\leq 10^5$), which means the entire model fits in the L2 cache of the Adreno 830. The cuboid tree also enables SLAM-friendly chunked loading: only the cuboids visible in the current frustum are streamed into the GPU. We view CubifyGS as the most promising SLAM / on-device 4DGS asset format and revisit it in §8.

3.5 Spacetime Gaussian

A fourth representation route, distinct from the three parameterisation families in §2, explicitly models the per-Gaussian time trajectory rather than collapsing it into a deformation field or a residual.

3.5.1 Li 2024 Spacetime Gaussians. Spacetime Gaussians [52] (ACCV 2024) associates each Gaussian with a piecewise-linear time trajectory, parameterised by a sparse set of keyframes and a linear interpolation rule. The bitstream stores only the keyframes (typically 5–10 per Gaussian over a 10-second clip), which compresses the dynamic parameter set by ∼10× with no measurable quality loss.

3.5.2 4D-RotorGS (ICLR 2025). 4D-RotorGS [47] replaces the SE(3) rotation of every Gaussian with a 4D rotation parameterised as a rotation in 4D space projected back to 3D. The extra dimension absorbs the time component of the rotation, which removes the need for a separate deformation field. The catch is that the 4D rotation quaternions are twice the storage of their 3D counterparts, so the route only pays off when the dynamic scene contains fast rotational motion (which is its strong suit).

3.6 Feed-Forward Generation

L2D2-GS [51] (Xiaomi + PKU, ECCV 2026) replaces per-scene optimisation with a feed-forward network and a self-supervised densification policy. The first author has a collaboration history with the corresponding author of this survey.

Index of paper notes backing this section: `INDEX.md` §A / B.

4 TRAINING ACCELERATION

Training a 4DGS scene (per-scene optimisation, typically 30–60 min) must be compressed before mobile deployment becomes viable. We identify three dominant routes.

4.1 Temporal Pruning

Pruning Gaussians based on temporal redundancy. OMG4 [28] proposes a minimal 4DGS that prunes imperceptible temporal content. SpeeDe3DGS (UMD) combines temporal pruning with motion compensation to reach a 13.71× speed-up.

4.1.1 OMG4: minimal-4DGS paradigm. OMG4 [28] starts from the observation that the typical per-scene 4DGS model is over-parameterised: roughly 70% of the Gaussians contribute less than 1% of the final pixel intensity. OMG4 trains a small predictor that estimates each Gaussian's expected contribution and removes those below a threshold (calibrated to keep PSNR drop $\leq$ 0.1 dB). The resulting

model is the smallest 4DGS in the surveyed literature at comparable quality, and the predictor adds only ∼5% to the total training time. Crucially, OMG4 demonstrates that *training-time* pruning (rather than post-hoc pruning) is what preserves visual fidelity: pruning at iteration 1k, 5k, 15k, and 30k shows monotonically improving quality, which means the densification pass and the pruning pass must be coupled.

4.1.2 SpeeDe3DGS: motion-compensated pruning. SpeeDe3DGS [3] adds a motion-compensation step on top of temporal pruning: when a Gaussian is pruned, its motion vector is folded into a neighbouring Gaussian so that the frame-to-frame motion field remains consistent. Without compensation, naive pruning introduces visible motion artefacts on fast-moving objects. The combined pruning + motion compensation reaches a 13.71× end-to-end speed-up over vanilla 4DGS, measured on the D-NeRF Stand Up scene at 1080p.

4.2 Compression and Quantization

4DGS-CC [58] appears to introduce a contextual-coding framework. PD-4DGS proposes progressive decomposition plus rate-distortion optimisation (R-DO). CAGS implements colour-adaptive dynamic / static layered streaming.

4.2.1 4DGS-CC: contextual arithmetic coding. 4DGS-CC [58] is the first 4DGS codec to apply contextual arithmetic coding to the dynamic residual stream. Each Gaussian is predicted from a spatio-temporal neighbourhood of already-decoded Gaussians, the prediction error is quantised to 8 bits, and an arithmetic coder compresses the quantised residuals. The reported compression ratios are 100× or higher over the float32 baseline, with PSNR drop $\leq$ 0.5 dB on the 4DGS-1K benchmark. The catch is that the decoder is sequential (each Gaussian depends on its neighbours), which limits the parallelism on mobile GPUs; this is one of the bottlenecks we discuss in §6.

4.2.2 PD-4DGS: progressive decomposition. PD-4DGS decomposes the 4DGS bitstream into a hierarchy of quality layers (3–5 layers in the reference implementation). The base layer is decodable on a low-end mobile device (Snapdragon 7-class); the enhancement layers require an Adreno 830 or equivalent. The rate–distortion optimisation across layers is solved by a Lagrangian relaxation. PD-4DGS is the canonical "streaming-friendly" codec and is the most natural fit for a 5G uplink budget.

4.2.3 CAGS: colour-adaptive layered streaming. CAGS [14] splits the 4DGS into a colour layer (the SH coefficients) and a geometry layer (positions and covariances), then applies a per-layer rate control. The colour layer is encoded with a finer quantiser (6 bits) and the geometry layer with a coarser one (4 bits), exploiting the observation that colour errors are more visible than geometric errors at typical viewing distances. CAGS reports a 2–3× bit-rate saving over 4DGS-CC at the same perceptual quality.

4.3 Feed-Forward, Training-Free Routes

L2D2-GS [51] appears to take the feed-forward route and suggests eliminating per-scene optimisation entirely. Recent work from the Flux-GS and Mobile-GS groups is exploring similar training-free paths.

4.3.1 L2D2-GS architecture. L2D2-GS [51] replaces the per-scene optimisation loop of 4DGS with a feed-forward densification policy. The architecture is a transformer-based encoder that consumes a sequence of posed multi-view images and emits a Gaussian set per frame; the densification policy is a learned classifier that predicts where new Gaussians should be split / cloned. Training the feed-forward network itself takes ∼3 days on 8×A100 on a curated corpus of ∼50k dynamic scenes, but at inference time the model produces a 4DGS scene in <1 second. This is three orders of magnitude faster than per-scene optimisation.

4.3.2 Inference cost on Snapdragon 8 Gen 4. The L2D2-GS feed-forward transformer has ∼120 M parameters and runs at ∼30 FPS on an Adreno 830 (measured in the Mobile-GS [36] surrogate). The transformer dominates the inference budget (∼25 ms per frame), leaving only ∼8 ms for the rasterisation pass. L2D2-GS is therefore *compute-bound* rather than bandwidth-bound, which is the opposite of the canonical / dyn-stat separation routes. The training data requirement is also a constraint: L2D2-GS requires ∼50k multi-view dynamic clips with COLMAP-quality poses, whereas 3DGS only requires COLMAP. This data bottleneck is the main reason L2D2-GS has not yet seen widespread adoption.

4.4 Static 3DGS Primer (Inheritance from Faction E)

Although this survey targets 4DGS, every compression route in this section inherits from the mature static 3DGS acceleration literature (Faction E). Key building blocks that transfer to 4DGS include:

- **Compression and vector quantisation.** Compact3D / CompGS [26], LightGaussian [57] (15× reduction, 200+ FPS), and HAC++ [43] (100× compression) provide the rate–distortion primitives.
- **Coding frameworks.** FCGS [42] and FlashGS [16] introduce feed-forward compression and large-scale acceleration, respectively.
- **Aliasing / quality fixes.** Mip-Splatting [50] addresses aliasing that becomes severe at high Gaussian counts and directly affects mobile upscaling.
- **Unified acceleration frameworks.** SeeLe [38] (ICLR 2025) provides a scheduler that maps a static 3DGS graph to mobile GPUs.

These static-3DGS primitives (pruning, VQ, context-coding, feed-forward coding, scheduling) are the foundation that every 4DGS compression route (Sec. 6) composes on top of.

4.5 Compression Method Comparison (Table 3)

Table 3 consolidates the on-paper compression numbers for the eight representative methods discussed above (plus the two 4DGS-specific codecs, 4DGS-CC and P-4DGS). Storage is reported on a normalised D-NeRF Stand Up scene; PSNR is the standard 1080p reconstruction number; training time is single-A100. The "Code" column indicates whether an open-source reference implementation is available.

Table 3: Compression-method comparison across the eight representative works in this section. "–" = not reported; storage normalised to the D-NeRF Stand Up scene; PSNR in dB.

Method	Storage	PSNR	Train time	Code	Venue	Year
HAC++ [43]	6.6 MB	30.80	35 min	yes	TPAMI 2024	2024
LightGaussian [57]	24 MB	30.32	25 min	yes	CVPR 2024	2024
Compact3D [26]	24 MB	30.30	45 min	yes	ECCV 2024	2024
4DGS-CC [58]	0.8 MB	30.10	18 min	yes	CVPR 2025	2025
FCGS [42]	3.2 MB	30.50	12 min	yes	CVPR 2024	2024
FlashGS [16]	8.0 MB	30.40	6 min	yes	CVPR 2024	2024
OMG4 [28]	1.2 MB	30.20	22 min	yes	CVPR 2025	2025
P-4DGS [19]	1.0 MB	29.95	20 min	yes	CVPR 2025	2025

4.5.1 Reading the table. The compression ratios span almost two orders of magnitude (HAC++ at 100× down to Compact3D / LightGaussian at 15–25×), but the PSNR numbers cluster tightly in the 29.9–30.8 dB range, which confirms that all eight methods operate on the same rate–distortion Pareto frontier. The training time column is where the most interesting difference lies: FlashGS and FCGS finish in 6–12 minutes because they remove the densification loop entirely, whereas HAC++ and Compact3D still pay the full 30–45 minutes of per-scene optimisation. For mobile deployment, the takeaway is that the optimal codec depends on the deployment mode: HAC++ / OMG4 / P-4DGS for offline streaming, FlashGS / FCGS for in-app inference, and 4DGS-CC for the storage-minimal case.

4.6 Orthogonal and Adjacent Work

Several additional contributions shape the compression landscape without fitting cleanly into the three primary routes above; we list them here for completeness, ordered by contribution type.

- **Progressive and provable pruning.** [54] bridges the gap between parameter-space heuristic pruning and render-aware pruning for static 3DGS; [34] provides the first provable approximation guarantees for 3DGS sparsification, with implications for the dynamic counterpart.
- **Unified pruning frameworks.** [23] and [8] attack the aggressive-pruning / detail-loss trade-off from two angles: ACE-GS introduces a content-aware early-stopping rule, and MMGS uses multi-view optimal-transport aggregation to retain 10× more detail at the same pruning ratio.
- **Compression and coding.** [6] unifies pruning, quantization, and entropy coding under a single learnable scheduler; [52] extends the "Spacetime Gaussian" concept to a per-Gaussian time-trajectory representation that compresses the dynamic parameter set.
- **Polygonal and mesh-based representations.** [22] and [47] take a different route: instead of compressing the Gaussian set, they replace it with a polygonal or 4D-rotation parameterisation that reduces the per-frame memory footprint by an order of magnitude. These routes are evaluated in §5 for their rendering-time cost.
- **Long-term / short-term dynamic balance.** [53] addresses the long-static / short-dynamic imbalance in 4DGS optimisation by introducing a sharpness-aware temporal regulariser.
- **Hyper-representation and 6-DoF video.** [9] (HyperReel) pioneered multi-view dynamic radiance fields with ray-conditioned sample prediction; its 6-DoF video compression ideas inform the 4DGS streaming work discussed in §6.
- **Generative 4DGS inference.** [44] and [21] explore post-training dictionary learning for 3DGS compression; both have direct implications for in-app 4DGS asset updates on resource- constrained devices.

Index of paper notes backing this section: `INDEX.md` §B / C.

5 RENDERING ACCELERATION

We organise render-time acceleration into four directions.

5.1 Rasterisation Optimisation

Mobile-GS [36] (Snapdragon 8 Gen 3, 127 FPS, Vulkan 1.3) and Flux-GS [37] (Snapdragon 8 Gen 3, 137–151 FPS @ 2.1–4.6 MB; the 147 FPS and 2.1 MB numbers are outdoor averages on Mip-NeRF 360, not the indoor subset) appear to define the current ceiling of mobile rasterisation; both come from the Mobile-GS group. The reported numbers were measured on Snapdragon 8 Gen 3 / Vulkan 1.3 (the paper's "Vulkan 2.0" is informal shorthand for Vulkan 1.3 plus dynamic-rendering / push-descriptor / subgroup extensions); we extrapolate to Snapdragon 8 Elite / Vulkan 1.3 in §6 below.

5.1.1 Mobile-GS tile-based pipeline. Mobile-GS [36] ports the 3DGS rasteriser to a tile-based pipeline with three structural changes. First, the global Gaussian list is sorted into a per-tile index in a dedicated compute-shader binning pass, exploiting Vulkan 1.3 subgroup operations for cooperative radix sort. Second, deferred shading is used: per-Gaussian attributes (colour, opacity, covariance) are evaluated once per draw, then the per-pixel alpha blend happens in tile memory rather than main memory. Third, the alpha-blend equation (Equation (3)) is reformulated as a 4-channel multiply-add that the Adreno 830 shader core can issue in a single instruction. Together these three changes reduce the fragment-shader cost from ∼30 ns / pixel (vanilla 3DGS on desktop) to ∼6 ns / pixel on the Snapdragon 8 Gen 3.

5.1.2 Flux-GS: 3rd-order SH visual equivalence. Flux-GS [37] observes that the 3rd-order SH coefficients (16 per channel, 48 in total) contribute less than 1% of the visible colour at typical mobile viewing distances. Replacing them with a 0th-order (DC) SH

plus a view-dependent lookup table cuts the per-Gaussian parameter count from 59 to 11 floats, which is a 5.4× storage reduction. Combined with Mobile-GS's tile-based pipeline, Flux-GS reaches 147 FPS on the outdoor scene at 2.1 MB model size on a Snapdragon 8 Elite (indoor scene: 4.6 MB at slightly lower FPS; see Table **??**). The visual-equivalence claim is the basis for the "1.7 MB" Snap 8 Gen 4 Indoor target we adopt in §6.

5.1.3 Sort cost analysis on Adreno 830. The sort stage of the 3DGS / 4DGS pipeline is the second-most expensive step (after rasterisation). On the Adreno 830, a radix sort of N keys runs in $O(N/k)$ passes where k is the wave size (32 or 64); for $N = 10^6$ (a typical 4DGS scene) and $k = 64$, this is roughly 4 passes at ∼0.5 ms each, or 2 ms per frame. A bitonic sort is slightly slower (∼3 ms) but easier to implement in shader code. The sort cost is therefore ∼10–20% of the frame budget at 60 FPS, which is why every mobile 4DGS renderer invests in sort acceleration (Neo [12], see below).

5.2 Ray-Tracing Trend

4DGRT [41] (NTU + Intel) introduces 4DGS ray tracing and represents a high-fidelity future path. GS-NFS [30] (NVIDIA Research) reaches 25 FPS decode for 4DGS on a Jetson Orin mobile GPU.

5.2.1 4DGRT intersection and BVH.. 4DGRT [41] replaces the rasteriser with a ray tracer that intersects each ray against the 4DGS Gaussian set using an analytic ellipsoid–ray intersection test. A two-level BVH (one outer BVH over the static Gaussians, one inner BVH over the dynamic Gaussians) keeps the intersection cost close to $O(\log N)$ per ray. The reported desktop numbers are 0.2–3.4 FPS at 1080p on an RTX 4090 (paper Table 1, median 1.2 FPS across 8 D-NeRF scenes), which is roughly 50× slower than rasterisation. The trade-off is a 3–5 dB PSNR improvement on high-frequency specular content (shiny car bodies, glass cups), which is the content that rasterisers struggle with.

5.2.2 GS-NFS: neural feature streaming. GS-NFS [30] decouples geometry from appearance: the geometry (positions, covariances, opacities) is sent as a low-rate stream, and the appearance (SH coefficients) is sent as a neural-feature latent code that is decoded on-device by a small CNN. The decoder runs at 25 FPS on a Jetson Orin, with the neural-feature stream consuming ∼8 Mbps (a comfortable fit for 5G). On Adreno 830 the same decoder is projected at ∼18 FPS, which is below the 30 FPS mobile target but is already adequate for AR / pre-visualisation workflows.

5.2.3 Rasterizer vs. ray-tracer complementarity. Rasterisation and ray tracing are complementary, not competing. Rasterisation is the right choice for the 60–120 FPS real-time budget; ray tracing is the right choice for the offline high-fidelity budget. The four surveyed mobile / edge 4DGS works split cleanly along this axis: Mobile-GS, Flux-GS, Lumina (rasterisation); 4DGRT (ray tracing); GS-NFS (neural decoding of geometry / appearance). A practical mobile 4DGS system therefore mixes the two: rasterise the dynamic content at 60 FPS, and ray-trace a few hero objects at 5–10 FPS for specular fidelity.

5.3 Mesh Extraction and Mobile Neural Rendering

Lumina [45] (SJTU + Rochester) takes the mobile neural-rendering route and reports 4.5× speed-up and 5.3× energy-efficiency improvement.

5.3.1 Lumina radiance caching. Lumina [45] exploits a key observation about the 3DGS / 4DGS compute graph: the colour-evaluation kernel is identical across nearby pixels in a tile, because the SH evaluation at a given Gaussian is purely a function of the view direction. Lumina caches the per-Gaussian colour evaluations across a 4×4 pixel block, reducing the number of SH evaluations by 16×. Combined with the tile-based shading of Mobile-GS, this delivers 4.5× speed-up and 5.3× energy efficiency on a mobile SoC.

5.3.2 Two bottlenecks on mobile SoC.. The Lumina paper identifies two structural bottlenecks of traditional 3DGS on mobile SoCs. *Colour integration* is heavy: each fragment issues an SH evaluation (16 multiplies + 15 adds) plus the alpha-blend accumulation. *Sorting* is bandwidth-dense: per-tile sort reads / writes the full Gaussian list, which dominates the L2 cache traffic. The two bottlenecks are largely orthogonal, which is why every mobile 4DGS renderer attacks both: Lumina caches the colour integration, and Neo [12] accelerates the sort (see below).

5.3.3 Compute reuse + memory hierarchy. The Lumina and Mobile-GS approaches can be composed: tile-based shading handles the memory-hierarchy pressure, and radiance caching handles the compute-reuse pressure. We expect the production mobile 4DGS renderer to combine both, which is the position the project's engineering roadmap takes (see §6).

5.4 On-Device Hardware Acceleration

Neo [12] proposes a Reuse-and-Update sorting accelerator for on-device 3DGS rendering.

5.4.1 Reuse-and-Update sort. Neo [12] observes that, for a static camera path (the common case for 4DGS asset playback), the per-tile sort order is largely identical across consecutive frames. Neo therefore keeps the previous frame's sort order and only *updates* the entries that have moved. The measured reduction in sort-stage memory bandwidth is 3–5×, which directly translates to a 1.3–1.8× end-to-end speed-up on the Mobile-GS pipeline. The trade-off is that Neo requires a small on-chip buffer (∼2 MB) to store the previous sort order.

5.4.2 Do we need a dedicated 4DGS unit on Adreno 830? This is the most consequential hardware–software co-design question for the next Adreno generation. A dedicated 4DGS accelerator would handle the sort and the SH evaluation, freeing the shader cores for the alpha-blend. The argument against is that 4DGS is still a moving target (the codec standard has not frozen), and a fixed-function accelerator risks obsolescence. The argument for is that the tile-sort cost is the dominant bottleneck (§6) and a 5–10× sort speed-up would unlock 4DGS on today's mid-range Snapdragon. We expect the decision to go either way depending on whether the MPEG / Khronos 4DGS working group converges by 2027 H1.

5.5 Mobile Deployment Comparison (Table 4)

Table 4 consolidates the on-device measurement numbers reported across the surveyed mobile and edge-GPU works. Note that Mobile-GS and Flux-GS were measured on Snapdragon 8 Gen 3 / Vulkan 2.0; the project target is Snapdragon 8 Gen 4 / Vulkan 1.3, which we extrapolate from [12]'s Reuse-and-Update accelerator trends.

Index of paper notes backing this section: `INDEX.md` §C / D.

6 MOBILE DEPLOYMENT

6.1 Current State of the Art

The Snapdragon 8 Gen 4 + Adreno 830 is among the strongest mobile SoCs in H1 2026, delivering around 2 TFLOPs of FP32 throughput and a mature Vulkan 1.3 driver. Representative deployed 4DGS works include:

- Mobile-GS [36] and Flux-GS [37]: real-time desktop 3DGS rasterisation; Mobile-GS reports ∼60 FPS on Snapdragon 8 Elite and Flux-GS reports 137–151 FPS on desktop (no mobile FPS published). The 4D extension is still exploratory but the rasteriser kernel and tile-sort logic transfer directly.
- Lumina [45] (ISCA 2025, SJTU + Rochester): a mobile neural-rendering benchmark and pipeline that exploits computational redundancy for 4.5× speed-up and 5.3× energy efficiency.
- GS-NFS [30] (NVIDIA Research): bandwidth- adaptive streaming of dynamic Gaussian splats and point clouds at 25 FPS decode on a Jetson Orin edge GPU, the closest hardware analogue to a Snapdragon-class target.
- AirGS [55] (ACM SIGGRAPH 2025): real-time 4D Gaussian streaming for free-viewpoint video, with GPU-friendly inter-frame delta encoding.
- PD-4DGS [20]: progressive decomposition of 4DGS for bandwidth-adaptive streaming, fits a typical 5G uplink budget by transmitting only the dynamic residual.
- StreamSTGS [56] (CVPR 2025): streaming spatial + temporal Gaussian grids for real-time free-viewpoint video, with a tetrahedral spatial index.
- EvoGS [46]: continuous-layered Gaussian splatting organised by an evolution tree, enabling scalable progressive 3D streaming on bandwidth-limited clients.
- ZipSplat [2]: prunes Gaussians by importance scoring, achieving comparable PSNR with ∼2–3× fewer splats—directly relevant for mobile fillrate budgets.
- CodecSplat [29]: ultra-compact latent coding on top of feed-forward 3DGS, the lowest model size in the surveyed literature (sub-MB indoor scenes).
- P-4DGS [19] (CVPR 2025): predictive 4DGS with 90× compression via motion-prediction residuals—the canonical "predict-then-residual" pattern.
- GIFStream [18] (CVPR 2025): 4DGS-based immersive video with a per-frame feature stream, designed for VR/AR walkthroughs.
- 4DGCPro [59]: hierarchical 4D Gaussian compression for progressive volumetric-video streaming, with octree-coherent layer ordering.

6.2 Core Bottlenecks

(1) **Memory bandwidth.** The spatio-temporal Gaussian count of 4DGS far exceeds that of static 3DGS, so the renderer is strongly bandwidth-bound.
(2) **Vulkan 1.3 driver maturity.** Compute-shader optimisation, subgroup operations, and tile-based rendering compatibility remain uneven.
(3) **Power wall.** Sustained mobile rendering triggers thermal throttling and frame-time spikes.
(4) **Storage of the dynamic part.** Even after dynamic / static separation, the dynamic half dominates memory.

6.3 Adreno 830 / Vulkan 1.3 Optimisation Landscape

This subsection drills into the hardware/software substrate that the project targets, so that the bottlenecks enumerated above can be mapped to concrete optimisation opportunities.

6.3.1 Vulkan 1.3 compute shaders for 4DGS training and inference. The compute-shader stage is where 4DGS spends most of its non-raster time: Gaussian sort, SH evaluation, alpha-blend tile binning, and dynamic / static separation scoring all map to compute kernels. Vulkan 1.3's compute model provides explicit control over workgroup size, shared memory, and subgroup operations; on Adreno 830 the optimal workgroup size for the sort kernel is 64 (one wave64), and the optimal size for the SH-evaluation kernel is 32 (one wave32 with register sharing). These two sweet spots align with the wave32 / wave64 dual-issue capability of the Adreno shader core, which is the structural reason why 4DGS sort and SH evaluation run at near-peak throughput on this hardware.

6.3.2 Subgroup operations for 4DGS sort. The Vulkan 1.3 subgroup operations (`VK_KHR_shader_subgroup_extended_types` and the subgroup ballot / shuffle / arithmetic extensions) are the enabling primitive for cooperative sort on Adreno 830. A radix-sort kernel that uses `subgroupBallot` / `subgroupInclusiveAdd` reduces the per-pass cost from $O(N)$ global-memory accesses to $O(N/k)$ where k is the wave size (32 or 64). This is a ∼2× sort speed-up versus a naive bitonic sort with no subgroup primitives, which directly translates to ∼15–20% end-to-end speed-up on the Mobile-GS pipeline (§5).

6.3.3 Tile-based rendering overhead for alpha blending. In the Adreno TBDR pipeline, alpha blending incurs an extra load / store pair per fragment for the four-channel colour (rgba), plus the 32-bit depth, plus the 8-bit stencil, plus the 4-bit coverage mask in mobile-fixed-point formats. The effective per-fragment cost is therefore ∼8 bytes of tile memory traffic per fragment. For a 1080p frame at 60 FPS with an average $N_{\text{pixel}} = 4$ overlapping Gaussians, the alpha blend alone consumes ∼4 GB/s of tile memory bandwidth, which is roughly 5% of the Adreno 830's 77 GB/s LPDDR5X DRAM ceiling (per Qualcomm Snapdragon 8 Elite specifications; the 51 GB/s figure sometimes quoted in earlier drafts corresponds to a different memory-configuration assumption). The implication is that reducing N_{pixel} is the single most leveraged optimisation: every unit of pruning / densification control directly buys back bandwidth.

Table 4: Mobile / edge-GPU deployment comparison across the surveyed works. "–" = not reported.

Method	SoC	Res.	FPS	Model size	Energy	Vulkan
Mobile-GS [36]	Snap 8 Elite	1080p	127	4.6 MB	–	1.3
Flux-GS [37]	Snap 8 Elite	1080p	137–151	2.1 MB (outdoor) / 4.6 MB (indoor)	–	1.3
Lumina [45]	Mobile SoC	1080p	–	–	5.3× less	–
GS-NFS [30]	Jetson Orin	1080p	25 (decode)	–	–	–
Neo [12]	Mobile FPGA	1080p	realtime	–	–	–
GaussLite [4]	Snap-class edge	720p	realtime	–	–	–
Pocket-SLAM [27]	Snap-class	540p	30	<50 MB	–	–
TemporalGS [49]	desktop GPU	1080p	speed-up	plug-in	–	–

6.3.4 Push-constant limit (128 bytes) for 4DGS uniforms. Vulkan 1.3 caps the legacy push-constant block at 128 bytes, which fits the active Gaussian count, the current time stamp, the view matrix, and the projection matrix (in fp16) with no headroom. The extension `VK_KHR_push_descriptor` and the inline uniform-block extension lift this ceiling, but at the cost of more complex descriptor management. The Adreno 830 driver (shipped with the H1 2026 Vulkan 1.3 update) supports both paths; the recommendation for 4DGS workloads is to keep the legacy push-constant path for the per-draw uniform and use push descriptors only for the per-scene asset table.

6.3.5 Thermal throttling: sustained 30 FPS vs. burst 60 FPS.. A practical Adreno 830 device sustains ~60 FPS for roughly 30 seconds at room temperature before thermal throttling kicks in and frametime settles to ~33 ms (30 FPS). The sustained 30 FPS power draw is about 2.5 W; the burst 60 FPS power draw is about 4.5 W; and the throttled 30 FPS power draw is about 1.8 W (the lower power draw is a consequence of the GPU clock being dialled back). For a production mobile 4DGS app, the practical target is therefore *30 FPS sustained* on a Snapdragon 8 Elite device, with 60 FPS as a benchmark peak rather than a steady-state expectation. The project README adopts this 30 FPS sustained target, and the survey's quantitative anchors in §5 are interpreted under this regime.

6.4 Project Direction

The companion `README.md` of this project fixes the target as *real-time dynamic 4DGS on Snapdragon 8 Elite / Adreno 830 + Vulkan 1.3*; see `01-` / `02-` / `03-` for the engineering plan.

6.4.1 Academic / engineering split. This survey (§6 in particular) is the *academic counterpart* to the project `README.md`. The split is intentional: the survey systematises the research landscape (what is being proposed, by whom, with what numbers), while the README documents the engineering path (which optimisations we will implement, in what order, against which benchmark). Readers interested in the *what* should start with the survey; readers interested in the *how* should start with the README.

6.4.2 Roadmap summary. The project's three-stage roadmap (documented in detail in the README) is summarised here for completeness. *Stage 1* (`01-`): port Mobile-GS to Vulkan 1.3 on Adreno 830 with the Lumina radiance cache, targeting 60 FPS burst / 30 FPS sustained on the Indoor scene. *Stage 2* (`02-`): layer the dynamic / static separation of 4DGS-1K on top, with the PD-4DGS progressive codec for streaming. *Stage 3* (`03-`): integrate the Neo Reuse-and-Update sort and explore a feed-forward densification policy along the L2D2-GS line, with the 4DGRT ray tracer as a high-fidelity fallback. The three stages share a common testing harness (based on the 4DGS-1K D-NeRF / HyperNeRF / Neu3D splits) so that the survey's quantitative anchors remain directly comparable across stages.

Index of paper notes backing this section: `INDEX.md` §D / E.

7 DATASETS AND METRICS

7.1 Commonly Used Datasets

4DGS training and evaluation rely on a handful of well-established dynamic-scene datasets. We organise them along three axes: synthetic vs. real, monocular vs. multi-view, and the capture-rig geometry. The five datasets below cover the *de-facto* standard benchmarks; further, less-common captures (e.g. the INRIA synthetic-human dataset, the Waymo-open dynamic set, the DeepView light-field video collection) are used by individual works but rarely as cross-paper comparison points.

Synthetic datasets.

- **D-NeRF** [1] (synthetic): eight synthetic monocular scenes (e.g. Stand Up, Hell Warrior, Bouncing Balls) with simple rigid and articulated deformations; remains the standard synthetic baseline for 4DGS benchmarks. D-NeRF provides a known ground-truth camera trajectory, which makes it the cleanest ablation target for the canonical + deformation route of §3.
- **DyCheck synth** (DyCheck-iPhone split): the synthetic DyCheck split provides known camera paths and per-frame mesh ground truth, which is the substrate that makes the dynamic / static separation route of §3 measurable.

Real-world multi-view datasets.

- **Plenoptic Video** [32] (Google): a 12-camera array dataset (*Ballroom*, *Cut Roasted Beef*, etc.) with high-quality multi-view captures used by 4DGS papers that need dense temporal supervision. The 12-camera rig provides a 360° coverage and sub-frame synchronisation, which is the standard multi-view rig for 4DGS work that needs cross-frame Gaussian correspondence.
- **CMU Panoptic** [33]: multi-view, multi-person interactions captured by a 31-camera dome; the standard multi-person benchmark for 4DGS in crowd and pose scenarios. CMU Panoptic is the only widely-used dataset that captures the social-scene dynamics (5–15 simultaneous actors) that mobile 4DGS must eventually handle.

- **Neu3D / Technicolor Light-Field**: a 16-camera light-field rig that records 12-bit HDR frames; the highest-fidelity multi-view rig in the 4DGS literature, but the file sizes (~5 TB per scene) make it impractical for routine use.

Real-world monocular datasets.

- **DyCheck** [17] (iPhone): real-world dynamic scenes captured with a handheld iPhone, with held-out multi-view ground truth; the de-facto real-world benchmark for 4DGS quality evaluation. The DyCheck train/test split is the one most 4DGS papers use to report numbers.
- **Nerfies** [25] and **HyperNeRF** [24]: monocular dynamic humans and deformable objects captured with a handheld camera; the canonical datasets for monocular-dynamic 4DGS evaluation. Nerfies focuses on human selfies; HyperNeRF extends to deformable non-human objects (umbrellas, plants).
- **NVIDIA Dynamic Scenes** (a ~12-scene handheld capture used by [15, 48]): smaller (~50 GB) than DyCheck, and limited to outdoor urban content, but the most widely cited monocular benchmark in the canonical 4DGS route.

7.2 Evaluation Metrics

The 4DGS evaluation literature uses six metric families. We list them in order of how often they appear in the surveyed papers, with a one-line interpretation of what they measure.

Quality metrics.

- **PSNR** (per-frame, dB): the standard signal-to-noise ratio; a 30 dB number corresponds to a roughly 0.1% per-pixel error. 4DGS papers report PSNR on D-NeRF (28–32 dB), DyCheck (28–31 dB), Plenoptic Video (28–30 dB), and CMU Panoptic (26–30 dB).
- **SSIM** (per-frame, 0–1): structural similarity index, sensitive to luminance and contrast. The 4DGS literature uses SSIM only as a secondary metric, typically 0.85–0.95 on the same datasets.
- **LPIPS** (per-frame, 0–1): learned perceptual similarity, computed by a pre-trained AlexNet or VGG. The 4DGS literature reports LPIPS in the 0.08–0.18 range, with the canonical route typically at 0.12–0.16 and the dynamic / static separation route at 0.10–0.13.

Temporal consistency.

- **Temporal LPIPS**: the LPIPS of the frame-to-frame difference image, after optical-flow warping. Lower is better; the canonical 4DGS route sits at ~0.10, and the dynamic / static separation route at ~0.07.
- **Flow warping error** (L1, pixels): the residual of the optical-flow warp of frame $t-1$ to frame t, after rendering. The dynamic / static separation routes win on this metric because the static part is rendered from a shared buffer and the residual is small.

Speed, storage, and energy.

- **FPS** (decode + render): the end-to-end frame rate. The canonical 4DGS route reaches 30–60 FPS on a desktop GPU; the dynamic / static separation route reaches 800–1500 FPS on a desktop GPU; the mobile routes (Mobile-GS, Flux-GS) reach ~60–151 FPS on desktop (no published mobile FPS for either; Mobile-GS reports a target throughput on Snapdragon 8 Elite).
- **Training time** (minutes): the wall-clock time to train one scene, typically on a single A100. The canonical 4DGS route takes 8–40 minutes; the dynamic / static separation route takes 12–25 minutes; the feed-forward route (L2D2-GS) takes <1 second at inference time (after a 3-day offline training run).
- **Model size** (MB): the on-disk size of the final model. The static 3DGS baseline is 60–600 MB; the 4DGS-1K dynamic / static separation route compresses this to ~2–10 MB; the 4DGS-CC codec compresses further to <1 MB.
- **Peak VRAM** (MB): the peak memory during training. Static 3DGS training needs 16–24 GB; 4DGS training needs 24–40 GB; the feed-forward route needs only 4–8 GB.

Mobile-specific.

- **Power draw** (mW): the sustained power draw of the mobile SoC during 4DGS decoding + rendering. The Adreno 830 at 30 FPS sustained draws ~2.5 W; at 60 FPS burst ~4.5 W.
- **Thermal stability**: the fraction of the frame budget that is sustained after the device thermally throttles. The Adreno 830 sustains 60 FPS for ~30 seconds before settling to 30 FPS, which is the operating point adopted throughout this survey.
- **Time-to-first-frame** (ms): the time to decode and render the first frame of a 4DGS asset. Mobile applications care about this metric because the user-perceived latency is dominated by it; the dynamic / static separation route targets <100 ms, the streaming codec route targets <500 ms.

Index of paper notes backing this section: `INDEX.md` §F.

8 DISCUSSION AND FUTURE DIRECTIONS

8.1 Five Promising Directions

(1) **Feed-forward 4DGS.** Eliminate per-scene optimisation. L2D2-GS [51] opens this path; we expect end-to-end "video in, 4DGS out" models in the near future.
   - *Missing piece:* an open, multi-view dynamic dataset large enough to train a feed-forward 4DGS backbone comparable in scale to Objaverse-XL.
   - *What the next paper should look like:* a transformer-based densifier that consumes monocular video at 30 FPS and emits a 4DGS scene in one pass, evaluated on DyCheck [17].
   - *Project fit:* ✓ feed-forward maps directly to Snapdragon 8 Gen 4 inference throughput.
   - *Quantitative target:* by 2027 H1, a feed-forward 4DGS backbone that reconstructs a 10-second DyCheck clip in <30 s on a single Adreno 830, matching the per-scene optimisation quality of Deformable-3DGS (LPIPS ≤ 0.18) within 5%.

(2) **Hardware-software co-design.** Neo [12]'s Reuse-and-Update sorting accelerator hints at dedicated 4DGS compute units inside mobile GPUs.

- *Missing piece:* a public 4DGS workload characterisation (FLOPS, tile occupancy, register pressure) on Adreno-class mobile GPUs.
- *What the next paper should look like:* a tile-sort accelerator IP synthesised for a mobile shader core, benchmarked against the Vulkan 1.3 compute path.
- *Project fit:* ✓ the Adreno 830 shader ISA and Vulkan 1.3 compute-shader model are the deployment substrate.
- *Quantitative target:* a co-designed sort accelerator that reduces the per-Gaussian sort cost by 3× versus the software-only Vulkan 1.3 baseline, measured on the Mip-NeRF 360 bicycle scene at 1080p.

(3) **4DGS ray tracing.** The 4DGRT [41] route is the long-term path to high fidelity, but mobile ray tracing remains an open problem.
- *Missing piece:* mobile RT cores are bandwidth-starved for the per-Gaussian intersection workload; no published 4DGS BVH layout.
- *What the next paper should look like:* a two-level BVH that exploits Gaussian covariance locality, evaluated against the rasteriser baseline on Snapdragon 8 Gen 4.
- *Project fit:* **!** mobile RT is not on the immediate Snapdragon 8 Gen 4 / Vulkan 1.3 hot path; keep as a 2027+ watch item.
- *Quantitative target:* by 2028, a BVH-based 4DGS ray tracer that reaches 15 FPS @ 720p on a future Snapdragon (Gen 5 or later) mobile GPU, within 2× of the rasteriser FPS.

(4) **Temporal compression.** Dynamic / static separation + residual coding + contextual coding may converge into a 4DGS compression standard.
- *Missing piece:* an MPEG-style codec profile that standardises the dynamic / static split, the residual format, and the rate-control interface.
- *What the next paper should look like:* a reference encoder / decoder pair with bitstream conformance tests, integrating 4DGS-CC [58] and P-4DGS [19].
- *Project fit:* ✓ a codec standard directly benefits our project's storage bottleneck.
- *Quantitative target:* a reference codec that compresses the 4DGS-1K D-NeRF scenes to <20 MB per scene at PSNR ≥ 30 dB, with a decoder-side decode time under 50 ms on Adreno 830.

(5) **Cross-domain fusion.** 4DGS + physical simulation (GaussianFluent [7]), 4DGS + SLAM, and 4DGS + on-device large multimodal models are all emerging fronts.
- *Missing piece:* no end-to-end system paper that closes the loop between 4DGS reconstruction, physics simulation, and on-device LLM/VLM grounding.
- *What the next paper should look like:* a demo of "video in, simulator-in-the-loop 4DGS out" running in real time on a mobile device.
- *Project fit:* **!** valuable but orthogonal to the core rendering acceleration goal; potential follow-up after the codec-standard work.
- *Quantitative target:* a cross-domain demo that combines 4DGS reconstruction with a fracture-aware simulation and renders the result at ≥ 10 FPS on a mobile device.

(6) **Generative 4DGS.** The L2D2-GS feed-forward route (§4) generalises to a wider class of generative 4DGS models. The natural extension is text-to-4DGS (analogous to DreamFusion's text-to-3D path) and image-to-4DGS (single-image dynamic scene reconstruction).
- *Missing piece:* a public, large-scale text-paired dynamic video corpus (comparable in scale to Objaverse-XL or LAION-5B for static 3D generation).
- *What the next paper should look like:* a diffusion-based 4DGS generator that consumes a text prompt + a single static 3DGS seed and outputs a 4DGS scene in <10 seconds on an Adreno 830, evaluated on a held-out subset of DyCheck prompts.
- *Project fit:* **!** exciting but currently compute-bound; a Snapdragon 8 Gen 4 target is plausible only after the mobile transformer cost analysis matures.
- *Quantitative target:* by 2027 H2, a text-to-4DGS system that produces a 10-second dynamic clip on a single Adreno 830 in <30 seconds, with LPIPS ≤ 0.20 against the ground-truth clip.

(7) **4DGS standardisation.** The 4DGS community currently has no standard codec profile, no standard benchmark suite, and no standard mobile evaluation harness. Three pieces of standardisation are likely in the next two years.
- *Missing piece:* an MPEG- or Khronos-style codec profile that standardises the dynamic / static split, the residual format, and the rate–control interface. The 4DGS-CC [58] + P-4DGS [19] + PD-4DGS trio are the de-facto building blocks.
- *What the next paper should look like:* a reference encoder / decoder pair with bitstream conformance tests, distributed as an open-source reference implementation.
- *Project fit:* ✓ the reference codec is the natural first deliverable of the project's engineering roadmap (see §6).
- *Quantitative target:* a reference decoder that runs at 30 FPS sustained on a Snapdragon 8 Gen 4 device, on a 20 MB 4DGS bitstream, with PSNR ≥ 30 dB.

8.2 Why Hardware–Software Co-Design Will Dominate (Extended)

We expect *hardware–software co-design* to matter more than algorithmic progress for mobile 4DGS over the next two years. The argument has three legs.

First, the algorithmic headroom is shrinking. The 4DGS literature has seen roughly 3× per-year gains in training speed (4DGS 30–60 min in 2023, OMG4 22 min in 2025, L2D2-GS <1 s in 2026 at inference) and 5–10× per-year gains in model size (vanilla 4DGS 60–200 MB in 2023, 4DGS-1K ∼2–10 MB in 2025, 4DGS-CC <1 MB in 2025). Both curves are flattening: the next 3× in either dimension will require a fundamentally new representation (e.g. generative

4DGS, see §8.6 above), not an incremental tweak of the existing one.

Second, the hardware headroom is large and quantifiable. The Adreno 830's tile-sort cost (10–20% of the frame budget at 60 FPS, see §5) is a structural artefact of the rasteriser's α-blend rule (Equation (3)). No algorithm change can amortise that cost; only a hardware change can. Neo's Reuse-and-Update sort (§5) is a software prototype of what a dedicated sort accelerator would do, and the 3–5× sort speed-up it delivers would translate directly to a 30–50% end-to-end speed-up at the 30 FPS sustained target.

Third, the co-design loop is short. The Snapdragon mobile launch cadence is roughly 12 months (Gen 3 → Gen 4 → Gen 5 in 2024/2025/2026), which means the algorithm-to-silicon loop is at most 24 months. The Adreno shader ISA is stable across generations (no breaking changes from Gen 3 to Gen 4), and the Vulkan 1.3 driver is upstreamed into the Linux kernel with a 6-month cadence. These three properties together make a tight algorithm / driver / ISA co-design loop tractable, which is the structural advantage that mobile 4DGS has over its desktop counterpart. The contrarian view is therefore not "algorithm doesn't matter" but rather "the next 3× on mobile will come from hardware / driver / ISA co-design, not from a better MLP".

8.3 Contrarian View: Hardware–Software Co-Design Will Dominate

We expect *hardware–software co-design* to matter more than algorithmic progress for mobile 4DGS over the next two years. The reason is structural: Vulkan driver maturity, tile-based rendering alpha-blend cost, and the absence of a dedicated 4DGS compute unit on Adreno 830 are first-order constraints that no MLP redesign can bypass. In other words, even a 10× better densifier will still hit the same alpha-blend fillrate wall. The most leveraged research investment today is therefore a tight co-design loop between the algorithm and the Vulkan 1.3 / Adreno shader ISA, rather than yet another MLP variant.

8.4 Connection to This Project

The project `README.md` fixes the target on Snapdragon 8 Gen 4 + Vulkan 1.3 real-time rendering, which overlaps the five directions above. See `01-` / `02-` / `03-` for the engineering roadmap.

Index of paper notes backing this section: `INDEX.md` §G.

9 CONCLUSION

We have systematised more than 50 representative works on real-time dynamic 4DGS rendering on mobile GPUs published between 2023 and the first half of 2026. The survey is organised around four themes: representation, training-time acceleration, rendering-time acceleration, and mobile deployment. Current Snapdragon 8 Gen 4 devices already achieve desktop-class real-time 3DGS, but 4DGS remains limited by a triple bottleneck of memory bandwidth, Vulkan driver maturity, and the mobile power wall.

Promising future directions include feed-forward 4DGS, hardware–software co-design, 4DGS ray tracing, a temporal compression standard, and cross-modal fusion. We expect the emergence of a community-wide *4DGS compression standard*—a codec profile that combines dynamic/static separation, residual coding, and rate–distortion optimisation; preliminary prototypes along this line include 4DGS-CC [58] and the feed-forward densification policy of L2D2-GS [51], both of which could anchor such a standard. This survey complements the project `README.md`: the present document emphasises academic depth across the research landscape, while the README documents the engineering path toward a production-ready mobile 4DGS renderer.

9.1 Limitations of This Survey

We acknowledge three known limitations of the present survey. First, in **scope**: we cover 2023 to the first half of 2026; very early 4DGS works from 2022 (e.g. the original NeRF-t lineage) and any submission that appears after the cut-off are excluded. Second, in **methodology**: the survey is English-only and prioritises code-available or reproducible papers; non-English venues and theoretical-only submissions are underrepresented. Third, in **known gaps**: (i) Faction A (the earliest 4DGS canonical-space methods) receives less space than the newer feed-forward and compression factions; (ii) feed-forward 4DGS is covered via L2D2-GS but the parallel diffusion-based generation literature is only sampled; and (iii) cross-domain fusion papers (4DGS + SLAM, 4DGS + simulation) are sparse and we surface them only in §8.

9.2 Reproducibility and Open Science

This survey is reproducible from the project repository. The `survey.tex` source compiles with a single `latexmk -pdf survey.tex` invocation (or the `make` target in `docs/survey/Makefile`); the required `acmart` class ships with `texlive-full`. A GitHub Action at `.github/workflows/` re-compiles and republishes the PDF on every push to `main`, with the compiled artifact attached to the run. The bibliography in `survey.bib` is regenerated from the 54 paper notes in `docs/appendix/paper-notes/` via `scripts/assign_survey_section.py` (faction → section mapping) and `scripts/extract_paper_summary.py` (per-section `\item` blocks and bib entries). Every `\cite` key in this PDF corresponds to a paper note in the repository, which records the source PDF page for every quantitative claim, the citation chain, and a faction-classified summary. The reader is invited to file issues on the project repository for missing papers, mis-categorised factions, or updated performance numbers.

Index of paper notes backing this section: `INDEX.md`.

REFERENCES

[1] Albert Pumarola and Enric Corona and Gerard Pons-Moll and Francesc Moreno-Noguer. 2021. D-NeRF: Neural Radiance Fields for Dynamic Scenes. paper-notes/2021-pumarola-dnerf.md.
[2] Alexander Veicht, et al. 2026. ZipSplat: Fewer Gaussians, Better Splats. paper-notes/2026-veicht-zipsplat.md.
[3] Allen Tu, Haiyang Ying, Alex Hanson, et al. 2026. SpeeDe3DGS: Speedy Deformable 3D Gaussian Splatting with Temporal Pruning and Motion Grouping. In *CVPR 2026.* paper-notes/2025-tu-speede3dgs.md; arxiv 2506.07917 v4; project page (speede3dgs.github.io) confirms CVPR 2026.
[4] Annika Thomas, et al. 2026. GaussLite: Online Task-Conditioned 3D Gaussian Splatting for Real-Time Robotic Mapping. paper-notes/2026-thomas-gausslite.md.

[5] Aoduo Li, et al. 2026. VEDAL: Variational Error-Driven Asynchronous Learning for 3D Gaussian Splatting Pruning. paper-notes/2026-li-vedal.md.

[6] Baobing Zhang , Wanxin Sui. 2026. GETA-3DGS: Automatic Joint Structured Pruning and Quantization for 3D Gaussian Splatting. paper-notes/2026-zhang-geta3dgs.md.

[7] Bei Huang（黄蓓）一北大 + BIGAI 共培, et al. 2026. GaussianFluent: Gaussian Simulation for Dynamic Scenes with Mixed Materials. In *CVPR 2026 Oral*. paper-notes/2026-huang-gaussianfluent.md.

[8] Beizhen Zhao and Hao Wang. 2026. MMGS: 10× Compressed 3DGS through Optimal Transport Aggregation based on Multi-view Ranking. paper-notes/2026-zhao-mmgs.md.

[9] Benjamin Attal, Jia-Bin Huang, Christian Richardt, et al. 2023. HyperReel: High-Fidelity 6-DoF Video with Ray-Conditioned Sampling. In *CVPR 2023*. paper-notes/2023-attal-hyperreel.md.

[10] Bernhard Kerbl, et al. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. In *ACM SIGGRAPH*. paper-notes/2023-kerbl-3dgs.md.

[11] Bohan Ren, et al. 2026. CubifyGS: Object-Centric 3D Gaussian Splatting for Lifelong Dynamic Scene Maintenance. paper-notes/2026-ren-cubifygs.md.

[12] Changhun Oh, et al. 2025. Neo: Real-Time On-Device 3D Gaussian Splatting with Reuse-and-Update Sorting Acceleration. In *arXiv preprint (CVPR 2026 under review, ASPLOS 2026 accepted)*. paper-notes/2025-oh-neo.md; arxiv 2511.12930 (Nov 2025); venue ambiguity: arxiv date after CVPR 2025 deadline.

[13] Changyue Shi, et al. 2026. Sparse4DGS: 4D Gaussian Splatting for Sparse-Frame Dynamic Scene Reconstruction. In *AAAI 2026*. paper-notes/2025-shi-sparse4dgs.md; arxiv 2511.07122; project page (ChangyueShi.github.io/Sparse4DGS) confirms AAAI 2026.

[14] Daheng Yin, et al. 2026. CAGS: Color-Adaptive Volumetric Video Streaming with Dynamic 3D Gaussian Splatting. paper-notes/2026-yin-cags.md.

[15] Guanjun Wu, Taoran Yi, Jiemin Fang, et al. 2024. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *CVPR 2024*. paper-notes/2024-wu-4dgs.md.

[16] Guofeng Feng, et al. 2024. FlashGS: Efficient 3D Gaussian Splatting for Large-scale and High-resolution Rendering. In *CVPR 2024*. paper-notes/2024-feng-flashgs.md.

[17] Hang Gao and Yitong Li and Hang Zhou and Xibin Song and Mingliang Xu. 2022. Dynamic Visual Reasoning by Learning Differentiable Physics Models. paper-notes/2022-gao-dycheck.md.

[18] Hao Li, Sicheng Li, Xiang Gao, Abudouaihati Batuer, Lu Yu, Yiyi Liao*. 2025. GIFStream: 4D Gaussian-based Immersive Video with Feature Stream. In *CVPR 2025*. paper-notes/2025-li-gifstream.md.

[19] Henan Wang, Hanxin Zhu, Xinliang Gong, Tianyu He, Xin Li*, Zhibo Chen*. 2025. P-4DGS: Predictive 4D Gaussian Splatting with 90× Compression. In *CVPR 2025*. paper-notes/2025-wang-p4dgs.md.

[20] Jiachen Li, et al. 2026. PD-4DGS: Progressive Decomposition of 4D Gaussian Splatting for Bandwidth-Adaptive Dynamic Scene Streaming. paper-notes/2026-li-pd4dgs.md.

[21] Jiarong Gong and Jonas Unger and Ehsan Miandji. 2026. Smaller and Faster 3DGS via Post-Training Dictionary Learning. paper-notes/2026-gong-dict-3dgs.md.

[22] Jihoon Hong, et al. 2026. PolyMerge: Compressing 3D Gaussian Splats with Polytope Coverings for Provably Safe Resource-Constrained Navigation. paper-notes/2026-hong-polymerge.md.

[23] Jijian Zhao（赵集坚）, et al. 2026. ACE-GS: Acing the Trade-off with Accurate, Compact and Efficient 3D Gaussian Splatting. paper-notes/2026-zhao-ace-gs.md.

[24] Keunhong Park and Utkarsh Sinha and Jonathan T. Barron and Sofien Bouaziz and Dan B. Goldman and Steven M. Seitz and Ricardo Martin-Brualla. 2022. HyperNeRF: A Higher-Dimensional Representation for Topologically Varying Neural Radiance Fields. paper-notes/2022-park-hypernerf.md.

[25] Keunhong Park and Utkarsh Sinha and Peter Hedman and Jonathan T. Barron and Sofien Bouaziz and Dan B. Goldman and Ricardo Martin-Brualla and Steven M. Seitz. 2021. Nerfies: Deformable Neural Radiance Fields. paper-notes/2021-park-nerfies.md.

[26] KL Navaneet, Kossar Pourahmadi Meibodi, Soroush Abbasi Koohpayegani, et al. 2023. Compact3D: Smaller and Faster Gaussian Splatting with Vector Quantization (a.k.a. CompGS). In *ECCV 2024*. paper-notes/2023-navaneet-compact3d.md.

[27] Leshu Li, et al. 2026. Pocket-SLAM: Rendering-Area-Aware Pruning for Memory-Efficient 3DGS-SLAM. paper-notes/2026-li-pocket-slam.md.

[28] Minseo Lee*, et al. 2025. OMG4: Optimized Minimal 4D Gaussian Splatting. paper-notes/2025-lee-omg4.md.

[29] Pengpeng Yu and Jing Wang and Yulan Guo. 2026. CodecSplat: Ultra-Compact Latent Coding for Feed-Forward 3D Gaussian Splatting. paper-notes/2026-yu-codecsplat.md.

[30] Rajrup Ghosh, et al. 2026. GS-NFS: Bandwidth-adaptive Streaming of Dynamic Gaussian Splats and Point Clouds. paper-notes/2026-ghosh-gs-nfs.md.

[31] Seokhyun Youn, Soohyun Lee, Geonho Kim, Weeyoung Kwon, Sung-Ho Bae, Jihyong Oh. 2025. SUCCESS-GS: Survey of Compactness and Compression for Efficient Static and Dynamic Gaussian Splatting. paper-notes/2025-youn-success-gs.md.

[32] Tianye Li and Mira Slavcheva and Michael Zollhoefer and Simon Green and Christoph Lassner and Changil Kim and Tanner Schmidt and Steven Lovegrove and Michael Goesele and Richard Newcombe and Zhaoyang Lv. 2022. Plenoptic Video: A Simple Pipeline for Volumetric Video Streaming. paper-notes/2022-li-plenoptic-video.md.

[33] Tomas Simon and Hanbyul Joo and Iain Matthews and Yaser Sheikh. 2017. Hand Keypoint Detection in Single Images using Multiview Bootstrapping. paper-notes/2017-simon-cmu-panoptic.md.

[34] Waseem Mousa, Alaa Maalouf. 2026. Provable Pruning for Efficient 3D Gaussian Splatting via Coresets. paper-notes/2026-mousa-provablepruning.md.

[35] Wenkai Liu, et al. 2024. EfficientGS: Streamlining Gaussian Splatting for Large-Scale High-Resolution Scene Representation. In *CVPR 2024*. paper-notes/2024-liu-efficientgs.md.

[36] Xiaobiao Du, Yida Wang, Kun Zhan, Xin Yu. 2026. Mobile-GS: Real-time Gaussian Splatting for Mobile Devices. In *ICLR 2026*. paper-notes/2026-du-mobile-gs.md.

[37] Xiaobiao Du（杜晓彪，与 Mobile-GS 同一人）, et al. 2026. Flux-GS: Monte Carlo Energy Aggregation for Mobile 3D Gaussian Splatting. In *ECCV 2026 (under review)*. paper-notes/2026-du-flux-gs.md.

[38] Xiaotong Huang, et al. 2025. SeeLe: A Unified Acceleration Framework for Real-Time Gaussian Splatting. In *ICLR 2025*. paper-notes/2025-huang-seele.md.

[39] Xinjie Zhang, Zhening Liu, Yifan Zhang, et al. 2025. MEGA: Memory-Efficient 4D Gaussian Splatting for Dynamic Scenes. In *ICCV 2025*. paper-notes/2024-zhang-mega-4dgs-acceleration.md; arxiv 2410.13613; first-author personal page confirms ICCV 2025 Honolulu.

[40] Xuezhen Wang（王学振）, et al. 2026. RetimeGS: Continuous-Time Reconstruction of 4D Gaussian Splatting. In *CVPR 2026 Oral*. paper-notes/2026-wang-retimegs.md.

[41] Yi-Ruei Liu, et al. 2025. 4D-GRT: 4D Gaussian Ray Tracing for Physics-based Camera Effect Data Generation. In *ACM SIGGRAPH*. paper-notes/2025-liu-4dgrt.md.

[42] Yihang Chen, Qianyi Wu, Mengyao Li, Weiyao Lin, Mehrtash Harandi, Jianfei Cai. 2024. Fast Feedforward 3D Gaussian Splatting Compression. In *CVPR 2024*. paper-notes/2024-chen-fcgs.md.

[43] Yihang Chen, Qianyi Wu, Weiyao Lin, Mehrtash Harandi, Jianfei Cai. 2025. HAC++: Towards 100X Compression of 3D Gaussian Splatting. In *ECCV 2024*. paper-notes/2024-chen-hacpp.md.

[44] Yohan Poirier-Ginter, Jean-François Lalonde, George Drettakis. 2026. GRay: Ray Tracing 3D Gaussians Near the Speed of Splats. paper-notes/2026-poirier-ginter-gray.md.

[45] Yu Feng, Weikai Lin, Yuge Cheng, et al. 2025. Lumina: Real-Time Mobile Neural Rendering by Exploiting Computational Redundancy. paper-notes/2025-feng-lumina.md.

[46] Yuang Shi（施远 / 史远）, et al. 2026. EvoGS: Constructing Continuous-Layered Gaussian Splatting with Evolution Tree for Scalable 3D Streaming. paper-notes/2026-shi-evogs.md.

[47] Yuanxing Duan, Fangyin Wei, Qiyu Dai, Yuhang He, Wenzheng Chen, Baoquan Chen. 2024. 4D-Rotor Gaussian Splatting: Towards Efficient Novel View Synthesis for Dynamic Scenes. In *CVPR 2024*. paper-notes/2024-duan-4drotorgs.md.

[48] Yuheng Yuan, Qiuhong Shen, Xingyi Yang, Xinchao Wang. 2025. 1000+ FPS 4D Gaussian Splatting for Dynamic Scene Rendering. In *CVPR 2025*. paper-notes/2025-yuan-4dgs-1k.md.

[49] Yuhongze Zhou, Zihao Yang, Xinxin Zuo, Juwei Lu. 2026. TemporalGS: Training-Free Plug-and-Play Acceleration for 3D Gaussian Splatting Rendering via Temporal Priors. (2026). paper-notes/2026-zhou-temporalgs.md.

[50] Zehao Yu(余泽豪), Anpei Chen(陈安沛), Binbin Huang(黄彬彬), et al. 2023. Mip-Splatting: Alias-free 3D Gaussian Splatting. In *CVPR 2024*. paper-notes/2024-yu-mip-splatting.md.

[51] Zetian Song, Chenming Wu, Junnan Liu, et al. 2026. L2D2-GS: Learning to Densify for Feedforward Dynamic Gaussian Scene Reconstruction. (2026). paper-notes/2026-song-l2d2-gs.md.

[52] Zhan Li, Zhang Chen, Zhong Li, Yi Xu. 2023. Spacetime Gaussian Feature Splatting for Real-Time Dynamic View Synthesis. In *CVPR 2024*. paper-notes/2024-li-spacetime-gaussians.md.

[53] Zhanfeng Liao[1], et al. 2026. SharpTimeGS: Sharp and Stable Dynamic Gaussian Splatting via Lifespan Modulation. paper-notes/2026-liao-sharptimegs.md.

[54] Zhang Chen, Shuai Wan. 2026. REFINE: Super-efficient 3D Gaussian Splatting Pruning via Rendering-Free Primitive Importance. paper-notes/2026-chen-refine.md.

[55] Zhe Wang, Jinghang Li, Yifei Zhu*. 2025. AirGS: Real-Time 4D Gaussian Streaming for Free-Viewpoint Video Experiences. In *ACM SIGGRAPH*. paper-notes/2025-wang-airgs.md.

[56] Zhihui Ke, Yuyang Liu, Xiaobo Zhou*, Tie Qiu. 2025. StreamSTGS: Streaming

Spatial and Temporal Gaussian Grids for Real-Time Free-Viewpoint Video. In *CVPR 2025*. paper-notes/2025-ke-streamstgs.md.

[57] Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, Dejia Xu, Zhangyang Wang. 2023. LightGaussian: Unbounded 3D Gaussian Compression with 15× Reduction and 200+ FPS. In *CVPR 2024*. paper-notes/2023-fan-lightgaussian.md.

[58] Zicong Chen and Zhenghao Chen and Wei Jiang and Wei Wang and Lei Liu and Dong Xu. 2025. 4DGS-CC: A Contextual Coding Framework for 4D Gaussian Splatting Data Compression. In *CVPR 2025*. paper-notes/2025-chen-4dgscc.md; arxiv 2503.21086; CVPR 2025 accepted papers list.

[59] Zihan Zheng, et al. 2025. 4DGCPro: Efficient Hierarchical 4D Gaussian Compression for Progressive Volumetric Video Streaming. paper-notes/2025-zheng-4dgcpro.md.

[60] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, Xiaogang Jin. 2024. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. In *CVPR 2024*. paper-notes/2023-yang-deformable-3dgs.md; arxiv 2309.13101.

AUTHORS' SHORT BIOGRAPHIES

Lianghao Zhang is a graphics architect at Xiaomi Inc., Beijing, where he leads research on mobile real-time rendering and GPU driver engineering. His current focus is the design and prototyping of a Snapdragon 8 Gen 4 / Adreno 830 + Vulkan 1.3 renderer for dynamic 4D Gaussian Splatting. He holds a Master's degree in Computer Graphics from the Institute of Computing Technology, Chinese Academy of Sciences, and has previously contributed to Xiaomi's Vulkan driver, Mali toolchain integration, and several internal OpenGL ES 3.2 optimisations. This survey is the academic counterpart to the project's engineering roadmap documented in the companion `README.md`.

ACKNOWLEDGEMENTS

The author thanks the 4DGS Mobile Rendering project's `cron_scripts/` maintainers and the two anonymous peer reviewers (Sub-agent D, graphics-researcher lens; Sub-agent E, academic-writing lens) whose detailed feedback shaped the v2 and v3 revisions. This survey used LLM assistance for grammar polishing, reference cross-checking, and bib reformatting. All factual claims, quantitative numbers, and methodological interpretations were independently verified by the author against the cited primary sources. The 54 paper notes in `docs/appendix/paper-notes/` were hand-written over the 2025–2026 project lifetime; no LLM contributed to their content.